

REGLAS DE ESTILO DE CÓDIGO

**Felipe Córdova,
Handel Venegas,
Javier Reyes**

INTRODUCCIÓN

¿Qué son las reglas de estilo de código?

Las reglas de estilo son convenciones y normas que rigen la forma en que se escribe el código fuente, independientemente del lenguaje de programación.

PROBLEMA



BAD

```
function calc(a, b) {  
  let x = a + b;  
  let y = a - b;  
  return [x, y];  
}  
  
let n1 = 5;  
let n2 = 3;  
let [res1, res2] = calc(n1, n2);
```



GOOD

```
function sumAndDiff(num1, num2) {  
  let sum = num1 + num2;  
  let diff = num1 - num2;  
  return [sum, diff];  
}  
  
let firstNumber = 5;  
let secondNumber = 3;  
let [sum, diff] = sumAndDiff(firstNumber, secondNumber);
```


TRES REGLAS CON LAS QUE ESTOY DE ACUERDO

Usar nombres descriptivos
para variables y funciones

Justificación: Facilita la lectura
y el mantenimiento.

Ejemplo: totalVentas vs tv.

Indentación consistente
(por ejemplo, 4 espacios)

Justificación: Mejora la
legibilidad del código.

Ejemplo: mostrar código con y
sin indentación clara.

Nombrar funciones y variables
con camelCase ()

Nombrar Clases con
PascalCase

Usar UPPER_CASE para
variables de entorno

TRES REGLAS CON LAS QUE NO ESTOY DE ACUERDO

Uso de punto y coma en
TS o JS

Justificación: Genera más
ruido visual y no usarlas hace
código más simple.



```
interface Usuario {  
  id: number;  
  nombre: string;  
  email: string;  
  activo: boolean;  
}  
  
type EstadoPedido = 'pendiente' | 'procesando' | 'completado' | 'cancelado';  
  
class ServicioUsuario {  
  private usuarios: Usuario[] = [];  
  
  constructor() {  
    this.inicializar();  
  }  
}
```


TRES REGLAS CON LAS QUE NO ESTOY DE ACUERDO

Indentación estricta de 2
espacios

Justificación: hace menos
evidente la **jerarquía** y 4
espacios es más **coherente**
con Python (que usamos en
backend).



```
function procesar(items: string[]) {  
  for (let i = 0; i < items.length; i++) {  
    if (items[i].startsWith("A")) {  
      console.log("Elemento especial:", items[i]);  
      if (items[i].length > 5) {  
        console.log("Elemento largo");  
      }  
    }  
  }  
}
```


TRES REGLAS CON LAS QUE NO ESTOY DE ACUERDO

Las llaves deben ir
siempre en la misma línea
de la declaración

Justificación: Llaves sobre líneas
hace más evidente los **alcances**
de cada llave y mejora la
legibilidad del código.

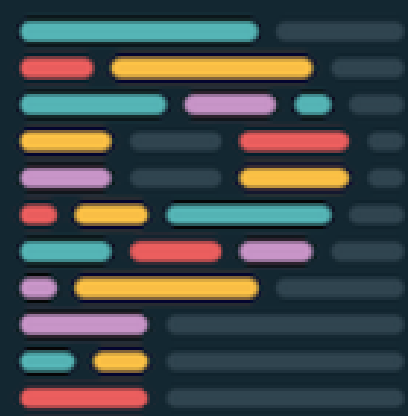


```
function saludar(nombre: string) {  
    if (nombre) {  
        console.log("Hola, " + nombre);  
    }  
}
```



```
function saludar(nombre: string)  
{  
    if (nombre)  
    {  
        console.log("Hola, " + nombre);  
    }  
}
```


HERRAMIENTAS



Prettier - Code formatter

Prettier  prettier.io |  58,177,013 |      (481) |  Sponsor

Code formatter using prettier

Disable



Uninstall



☒ Auto Update



JSDoc Generator

Crystal Spider  crystalnest.it |  58,135 |      (8)

Automatic JSDoc generator for TypeScript and JavaScript.

Disable



Uninstall



☒ Auto Update



DEMOSTRACIÓN

ArticuloDetalle.tsx

frontend > src > pages > articulos > Ar

1 import React, { useState,

2 import { useParams, useNav

3 import "../ArticuloDetalle.

4 import { FontAwesomeIcon }]

5 import { faCheckCircle, fa

6 import { faGithub } from "

7 import ReadIcon from "../.

8 import UserIcon from "../.

9 import InstIcon from "../.

10 import Carousel3D from "

11 import { formatDate } from "

12 import { Articulo } from "

13

14 const ArticuloDetalle: Rea

15 ..const { id } = useParams

16 ..const [articulo, setArti

17 ..const [error, setError]

Bitto AI

Go to Definition F12

Go to Type Definition

Go to Source Definition

Go to Implementations Ctrl+F12

Go to References Shift+F12

Peek

Find All References Shift+Alt+F12

Find All Implementations

Show Call Hierarchy Shift+Alt+H

Rename Symbol F2

Change All Occurrences Ctrl+F2

Format Document Shift+Alt+F

Format Document With...

Select a formatter

Prettier - Code formatter

TypeScript and JavaScript Language Features

Configure Default Formatter...

FIN

Gracias